Cuisine Prediction from Recipe Ingredients

Comprehensive Analysis using Scratch Implementations & Ensemble Learning

Anant Gupta (2025AIB2556)
Meghender Pal (2025AIB2563)

AIL 7024: Machine Learning
Department of Yardi School of AI

November 28, 2025

1. Problem Statement

Objective

To classify a recipe into one of **20 Cuisines** (e.g., Indian, Mexican) based purely on its text list of ingredients.

Evaluation Metric: We used **Accuracy** to evaluate our models.

$$\text{Accuracy} = \frac{\text{True Positives} + \text{True Negatives}}{\text{Total Samples}}$$

2. Dataset Overview

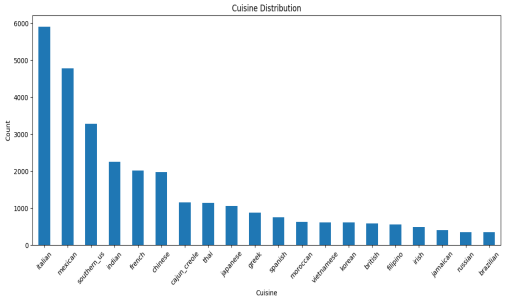
Metric	Value
Total Recipes	**29,774**
Number of Classes	20 Unique Cuisines
Input Format	JSON List of Strings
Target Variable	Categorical (e.g., 'greek', 'indian')

Split Strategy

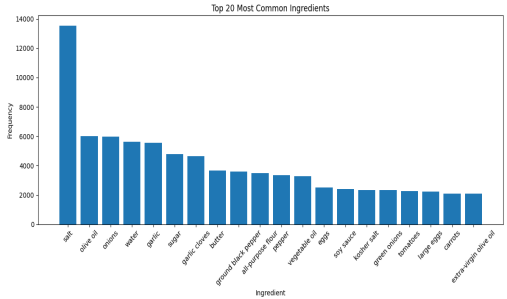
We split the data into **75% Training** ($\approx$ 22,330) and **25% Testing** ($\approx$ 7,444).

3. EDA: The Class Imbalance

- **Dominant Classes:** Italian (∼**6000**), Mexican (∼**5000**).
- **Minority Classes:** Brazilian, Jamaican (∼400).



4. EDA: Most Frequent Ingredients

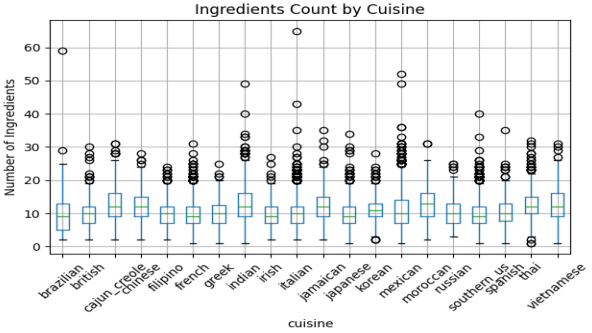


Observations:

- **Stop Words of Cooking:** Salt, Onions, Olive Oil, Water, Garlic.
- These ingredients appear in almost every cuisine.
- **Action:** These need to be down-weighted using TF-IDF.

5. EDA: Cuisine Complexity (Box Plot)

We analyzed the distribution of ingredient counts across cuisines.



Key Insights:

- **Complex:** Moroccan & Indian recipes have higher median ingredients ($\sim$12-15).
- **Simple:** Irish & British are simpler ($\sim$8-9).
- **Outliers:** Unique recipes with unusual complexity (30+ ingredients).

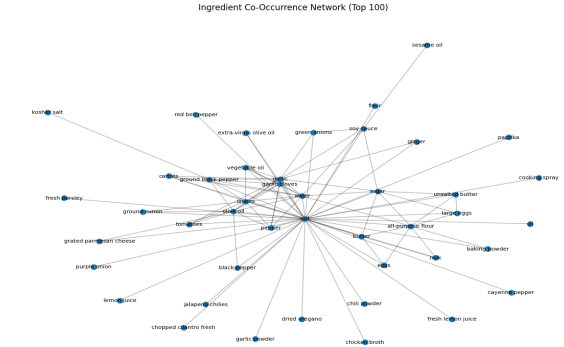
6. EDA: Visualizing Cuisine Signatures



Word Clouds highlight signature ingredients.

7. EDA: Ingredient Co-occurrence Network

Visualizing which ingredients are frequently cooked together.



Network Insights:

- **Hubs:** Garlic, Onions, and Oil form the central clusters.
- **Clusters:** Distinct clusters visible for baking (Flour, Sugar, Eggs).

8. The Custom Tokenizer

We built a custom Tokenizer class.

Preprocessing Logic

1. **Normalization:** Lowercasing all text.
2. **Phrase Merging:** "Kalamata Olives" $\rightarrow$ `kalamata_olives`.
3. **Symbol Removal:** Removed punctuations and digits (e.g., "1oz", "20g").

Filtering Strategy:

- `min_df=5`: Removes typos/rare brands.
- `max_df=0.9`: Removes ubiquitous words (Salt).

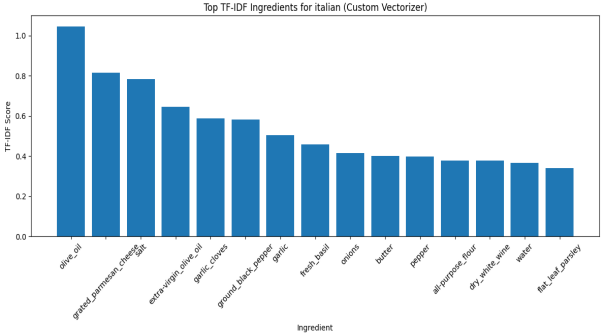
9. Feature Extraction: TF-IDF

1. **TF-IDF Vectorization**
   - **Term Frequency (TF):** How often an ingredient appears in a dish.
   - **Inverse Document Frequency (IDF):** How rare the ingredient is globally.
   - *Result:* "Wasabi" gets a high score; "Salt" gets a near-zero score.

2. **Feature Fusion**
   - Combined **Sparse Matrix (Text)** + **Numerical Feature (Length)**.
   - Final Shape: **(29774, 2946)**.

10. Feature Analysis: Top TF-IDF Ingredients

Does TF-IDF actually work? Let's check the highest weighted features.



Validation:

- **Indian:** *Cumin, Turmeric* (Correct).
- **Italian:** *Olive oil, Cheese* (Correct).
- **Noise Removed:** 'Salt' and 'Water' are successfully down-weighted.

11. Hero Model: Support Vector Machine (SVM)

- **Algorithm:** SVC with RBF Kernel.
- **Rationale:** Text data is high-dimensional and sparse. SVMs are mathematically best suited to find margins in such spaces.
- **Hyperparameters:**
  - $C = 2$ (Balanced Regularization).
  - *Gamma* = *Scale*.

Result

77.8%
(Test Accuracy)

12. Logistic Regression (Scratch Implementation)

Implemented from scratch using **Softmax** & **Gradient Descent**.

Key Equations:

- **Softmax Function:**

$$P(y = k|x) = \frac{e^{z_k}}{\sum_j e^{z_j}}$$

- **Update Rule:**

$$w \leftarrow w - \eta \cdot (y_{pred} - y_{actual}) \cdot x$$

Result

Test Accuracy: **77.7%**
(Matched SVM performance)

13. Decision Tree (Scratch Implementation)

Built a recursive tree structure using **Gini Impurity**.

Splitting Criteria:

- We minimize Gini Impurity at each split.
- **Formula:**

$$Gini = 1 - \sum_{i=1}^{C} (p_i)^2$$

- **Parameters:** `max_depth`, `min_samples_split`.

Logic:

- Iterates through all Approx 3000 features.
- Finds the best split that maximizes Information Gain.

14. Why did the Decision Tree Fail?

Result

Test Accuracy: 44.5%

Analysis:

- **Curse of Dimensionality:** With ≈3000 features, a single tree struggles to find meaningful splits without overfitting.
- **Verdict:** Single trees are weak learners for sparse text data.

15. The Solution: Random Forest (Bagging)

Since the Single Decision Tree failed (44%), we tried **Bagging**.

- **Hypothesis:** Aggregating hundreds of weak trees (Forest) reduces variance and improves accuracy.
- **Technique:** Bootstrap Aggregating (Bagging).
- **Setup:** Trained 300 trees in parallel and took a majority vote.

16. Random Forest Optimization Results

Hyperparameter Tuning (Grid Search):

- **Best Params:** `n_estimators=300`, `max_depth=None`.

Result

- **Test Accuracy: 70.0%**

Verdict:

- Massive improvement over Single Tree (44% $\rightarrow$ 70%).
- Proves that Ensemble methods work, though Linear models are still better for this data.

17. Naive Bayes (Scratch Implementation)

Implemented Multinomial Naive Bayes using Log-Probabilities.

Mathematical Foundation

$$\hat{y} = \underset{y}{\text{argmax}} \left(\log P(y) + \sum_{i=1}^{n} \log P(x_i|y) \right)$$

- **Laplace Smoothing:** Added $\alpha = 10$ to handle zero probabilities.
- **Result: 74.8%** Accuracy.

18. XGBoost (eXtreme Gradient Boosting)

- **Technique:** Sequential Boosting (Corrects previous errors).
- **Preprocessing:** Label Encoding for targets.
- **Hyperparameters:** `n_estimators`=200, `max_depth`=4.

Result: 72.8% (Lower than Linear Models).

19. LightGBM (Leaf-Wise Growth)

- **Strategy:** Leaf-wise growth (focuses on max error) vs Level-wise.
- **Tuning:** Tested `max_depth = 100` and `LR = 0.05`.
- **Performance:** Significantly faster than XGBoost.

Result

75.7%
(Better than XGBoost)

20. Final Model Comparison (Leaderboard)

Rank	Model	Type	Test Accuracy
1	**SVM (RBF)**	**Linear**	**77.8%**
2	Logistic Regression	Linear	77.7%
3	LightGBM	Boosting	75.7%
4	Naive Bayes	Probabilistic	74.8%
5	XGBoost	Boosting	72.8%
6	Random Forest	Bagging	70.0%
7	Decision Tree	Tree	44.5%

21. Why did Linear Models Win?

- **Data Nature:** TF-IDF creates a High-Dimensional (Approx 3000) and Sparse Matrix.
- **Linear vs Trees:**
  - **Trees** struggle to split space efficiently when features are sparse.
  - **Linear Models (SVM/LogReg)** effectively find a hyperplane to separate classes in high-dimensional space.

This explains why SVM (77.8%) beat Random Forest (70.0%).

22. Challenges Faced

1. Class Imbalance

Dominant Classes: Italian (∼**6000**), Mexican (∼**5000**).

Minority Classes: Brazilian, Jamaican (∼400).

2. Ingredient Overlap

Cajun vs *Southern US* share spices like Cayenne and Garlic, making separation difficult.

3. Sparsity

The "Curse of Dimensionality" negatively impacted tree-based models.

23. Future Improvements

- **N-Grams:** Using character or word n-grams instead of only unigrams to capture multi-word ingredients better.
- **Handling Imbalance:** Employing class-weighted loss functions or resampling strategies to mitigate class imbalance.
- **Deep Learning:** Exploring embedding-based models (e.g., Word2Vec, fastText) or deep learning architectures for further performance gains.

Thank You!

Questions?

Project by Anant Gupta & Meghender Pal